Lab #8: AI Agentic Lab

Simple Blogger Agent with Google's Agent Development Kit (ADK)

Anthony Bayate | Web Client-Side Programming | Kean University, Fall 2026

Summary: how the agentic lab works

The lab follows the YouTube tutorial and builds a small multi-agent system that turns a one-line topic into a finished technical blog post. Instead of one big prompt, the job is split between agents with narrow jobs, and each draft is checked before it moves on. Everything runs on Gemini through Google's ADK, and *adk web* gives a browser UI that shows every agent call, the shared state, and a timing trace.

Agent	Role	What it does
Blogger (root *Agent*)	Conductor	Reads the user's topic, calls *planner_tool*, then *writer_tool*, then adds 3 alternate titles and 2 tweet-length hooks.
RobustBlogPlanner (*LoopAgent*, max 3)	Planner	Runs BlogPlanner (writes *blog_outline*), then OutlineValidationChecker (writes *validation_result*), then StopIfOutlineOk.
RobustBlogWriter (*LoopAgent*, max 3)	Writer	Runs BlogWriter (reads *blog_outline*, writes *blog_post*), then BlogPostValidationChecker, then StopIfPostOk.
AgentTool wrappers	Glue	*agent_tool.AgentTool(agent=...)* turns each loop into a function the root model can call, the same way it would call any other tool.

A request flows like this:

- The user types a topic. The root **Blogger** agent doesn't write anything itself. Its instruction tells it to call the planner tool and then the writer tool.
- **Planning loop.** BlogPlanner returns a Markdown outline (title, intro, 4-6 sections, conclusion) and ADK saves it in session state under *blog_outline* because of *output_key*. OutlineValidationChecker reads that state and answers "ok" or "retry" plus what is missing. If the answer is "retry", the LoopAgent runs the planner again, up to 3 times.
- **Writing loop.** BlogWriter turns *blog_outline* into a full article for software engineers (*blog_post*). BlogPostValidationChecker checks it for an intro, sections that match the outline, a conclusion, and technical clarity, with the same ok/retry loop.
- **Finish.** Control returns to Blogger, which shows the article and adds 3 alternate titles and 2 tweet-length hooks.

What makes it "agentic" rather than one prompt: the root model decides which tool to call and when; the agents talk to each other through shared session state instead of passing long text around; and the checker agents make a feedback loop, so the system can catch and fix its own weak output. My run shows this. Asking for the "top ten" use cases made the planner write 10 sections, which breaks the checker's 4-6 section rule. The checker answered "retry", BlogPlanner ran again with fewer sections, the checker said "ok", and the loop stopped (see the Traces and loop-test screenshots).

Changes I made to the provided code

- The *agent.py* in the zip used plain *sub_agents* on the root agent, which is not the design in the tutorial. I rebuilt it to match the slides: two *LoopAgent*s wrapped with *agent_tool.AgentTool* and passed to the root agent as *tools*.

- Added a tiny *StopIfOk* agent at the end of each loop. The slides never show how a loop stops after "ok", so without it every loop always runs all 3 times. StopIfOk reads *validation_result* and sends an *escalate* event, which is how ADK ends a LoopAgent early.
- Inserted *{blog_outline}* / *{blog_post}* into the checker and writer instructions so the state values are actually put into the prompt.

Setup

I extracted *simple_blogger_agent_works.zip* into *week 8/lab8-blogger-agent*. The *.venv* inside the zip pointed to another person's Python install, so I made a new Python 3.12 virtual environment and installed *uv*, *google-adk* and *python-dotenv*, as in the tutorial's "Environment setup" slide.

Windows PowerShell - environment setup

```
PS> py -3.12 -m venv .venv
PS> .venv\Scripts\python -m pip install --upgrade pip uv
PS> .venv\Scripts\uv pip install google-adk python-dotenv
Checked 2 packages in 8ms

PS> .venv\Scripts\python --version
Python 3.12.10
PS> .venv\Scripts\adk --version
adk, version 2.12.0
PS> pip list | findstr "google-adk google-genai dotenv"
google-adk                    2.12.0
google-genai                  2.32.0
python-dotenv                 1.2.4
```

Creating the virtual environment and installing google-adk (ADK 2.12.0).

Windows PowerShell - project files

```
PS> tree /F  (project layout, .venv omitted)
lab8-blogger-agent
|-- .venv\
|-- smoke_test.py
|-- build_report.py
`-- simple_blogger_agent\
    |-- __init__.py
    |-- agent.py
    |-- requirements.txt
    |-- README.md
    |-- .env.example
    |-- .env

PS> type simple_blogger_agent\.env   (key redacted)
GOOGLE_API_KEY="AQ.Ab8***************REDACTED"
MODEL=gemini-flash-lite-latest

PS> type simple_blogger_agent\__init__.py
"""Simple Blogger Agent package."""

from .agent import root_agent

__all__ = ["root_agent"]
```

Project layout and .env config (API key redacted). __init__.py exposes root_agent to ADK.

Windows PowerShell - agent.py

```
PS> type simple_blogger_agent\agent.py   (excerpt: loops, tools, root agent)
class StopIfOk(BaseAgent):
    """Ends the surrounding LoopAgent once the checker says "ok".

    The slides only show the planner and checker inside each loop; without an
    escalate event the LoopAgent would always run all max_iterations.
    """

    async def _run_async_impl(self, ctx: InvocationContext) -> AsyncGenerator[Event, None]:
        result = str(ctx.session.state.get("validation_result", "")).strip().strip('"').lower()
        yield Event(author=self.name, actions=EventActions(escalate=result.startswith("ok")))


robust_blog_planner = LoopAgent(
    name="RobustBlogPlanner",
    description="Retries planning if validation fails.",
    sub_agents=[blog_planner, OutlineValidationChecker(), StopIfOk(name="StopIfOutlineOk")],
    max_iterations=3,
)
...
robust_blog_writer = LoopAgent(
    name="RobustBlogWriter",
    description="Retries writing if validation fails.",
    sub_agents=[blog_writer, BlogPostValidationChecker(), StopIfOk(name="StopIfPostOk")],
    max_iterations=3,
)


# Expose planner/writer as tools so the root agent can call them explicitly
planner_tool = agent_tool.AgentTool(agent=robust_blog_planner)
writer_tool = agent_tool.AgentTool(agent=robust_blog_writer)
```

agent.py excerpt: the two LoopAgents, the StopIfOk exit step, and the AgentTool wrappers.

Running and testing the agent

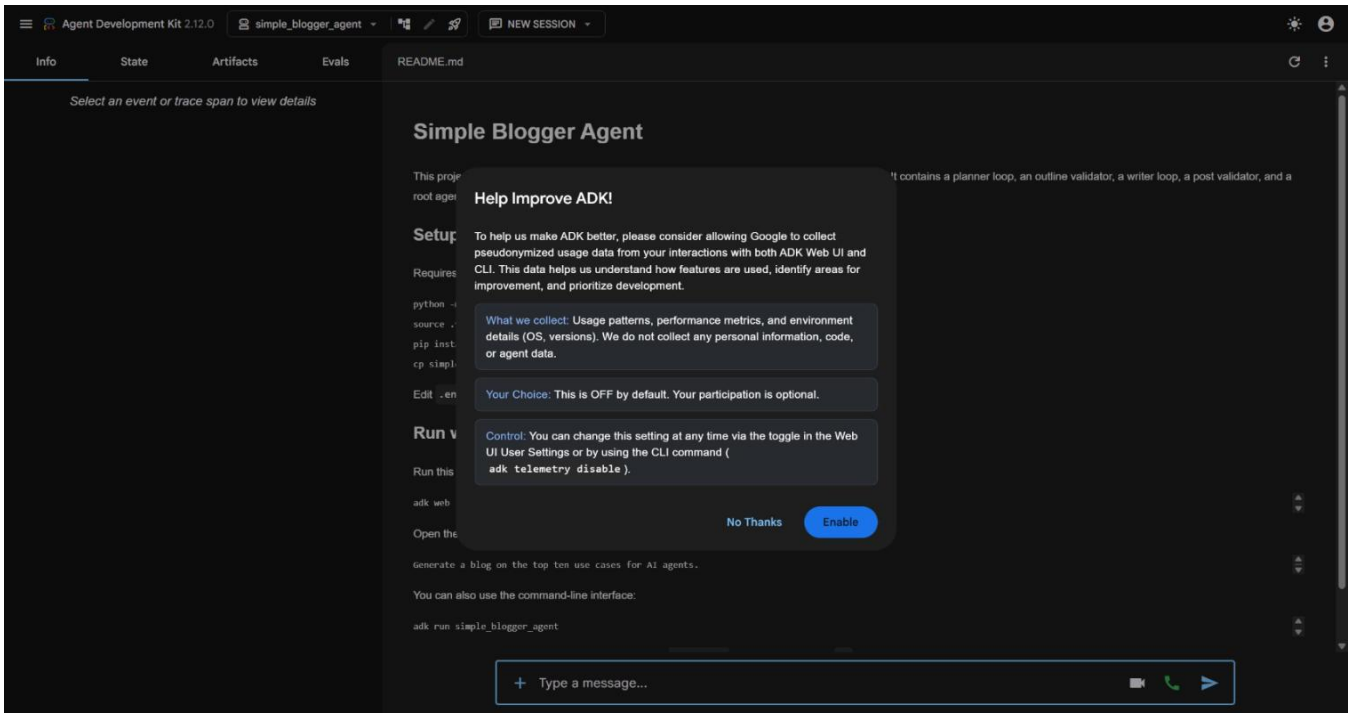
```
Windows PowerShell - adk web

PS> .venv\Scripts\adk web --port 8765 .
INFO:     Started server process [49968]
INFO:     Waiting for application startup.

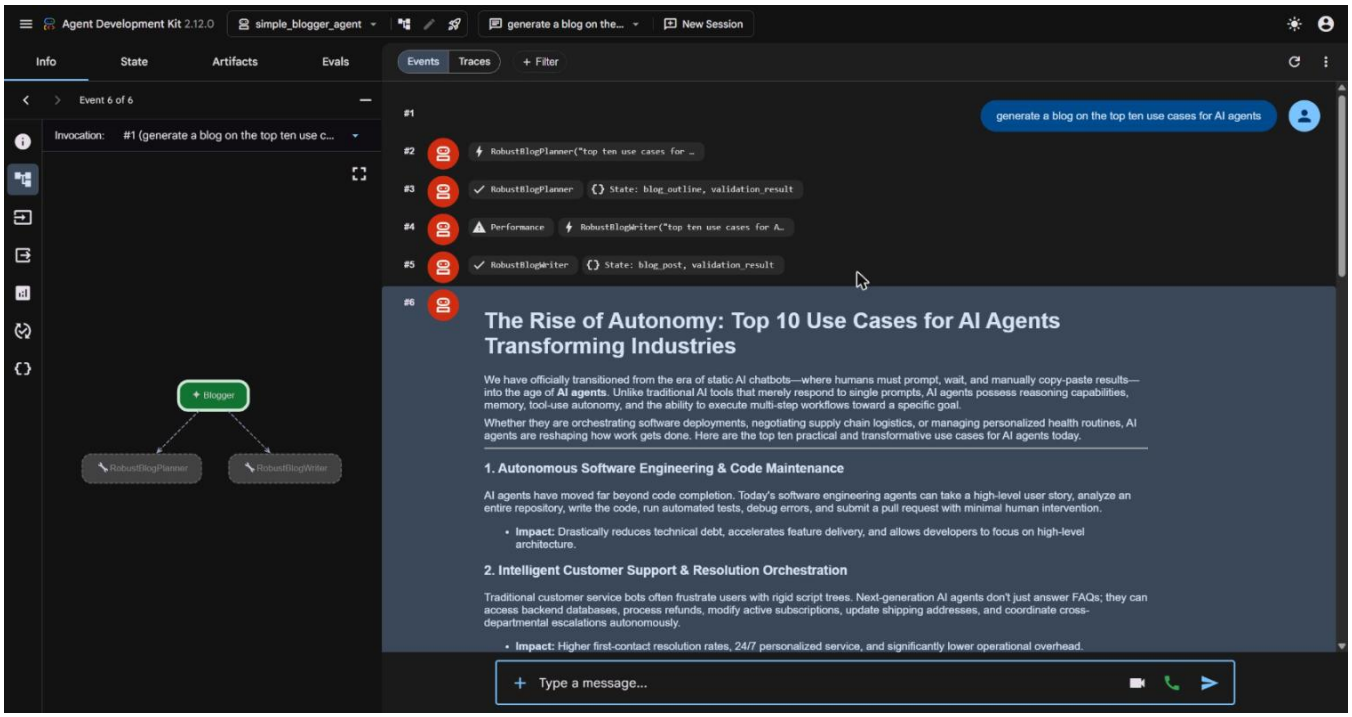
+-----------------------------------------------------------------------------+
| ADK Web Server started                                                      |
|                                                                             |
| For local testing, access at http://127.0.0.1:8765.                        |
+-----------------------------------------------------------------------------+

INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8765 (Press CTRL+C to quit)
INFO:     127.0.0.1:31926 - "POST /config/telemetry HTTP/1.1" 200 OK
INFO:     127.0.0.1:31926 - "POST /apps/simple_blogger_agent/users/user/sessions HTTP/1.1" 200 OK
INFO:     127.0.0.1:31926 - "POST /run_sse HTTP/1.1" 200 OK
15:15:11 HTTP POST gemini: gemini-flash-lite-latest:generateContent "HTTP/1.1 200 OK"
15:15:15 HTTP POST gemini: gemini-flash-lite-latest:generateContent "HTTP/1.1 200 OK"
15:15:16 HTTP POST gemini: gemini-flash-lite-latest:generateContent "HTTP/1.1 200 OK"
```

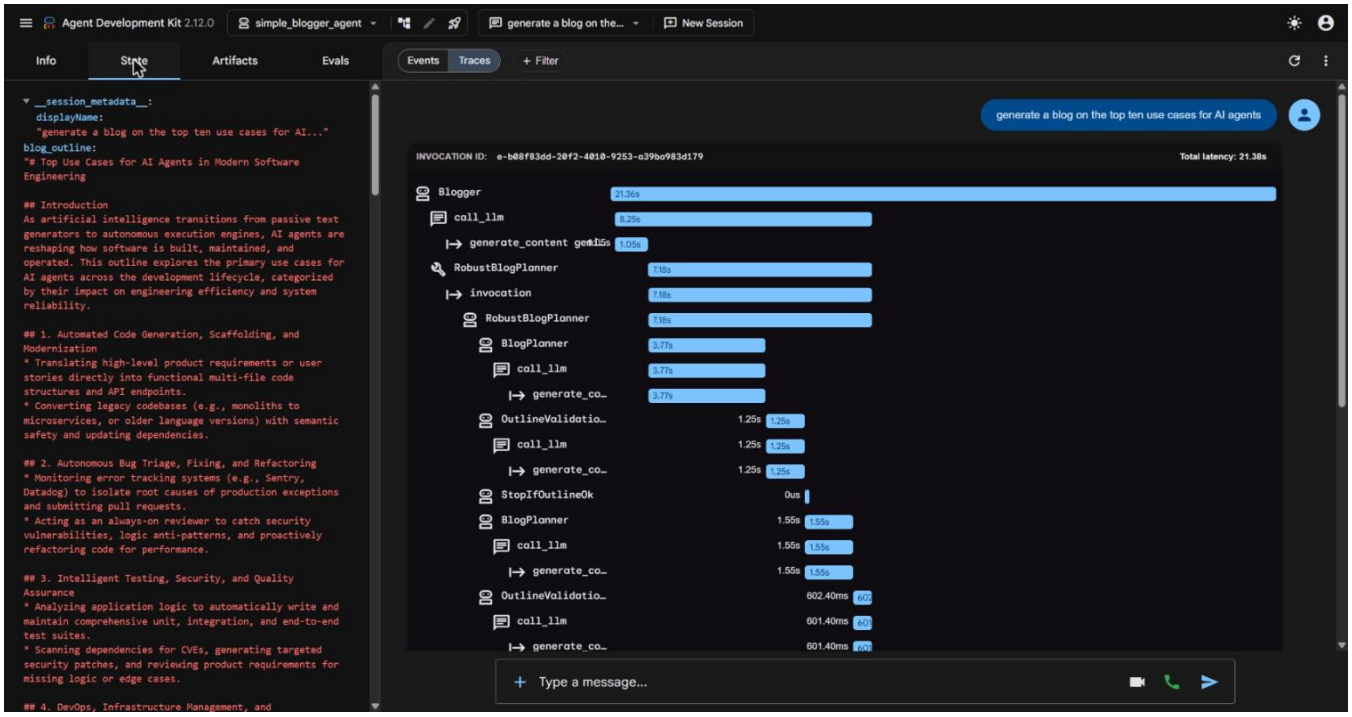
Starting the ADK web server (port 8765, since 8000 was in use) and the agent's calls to Gemini.



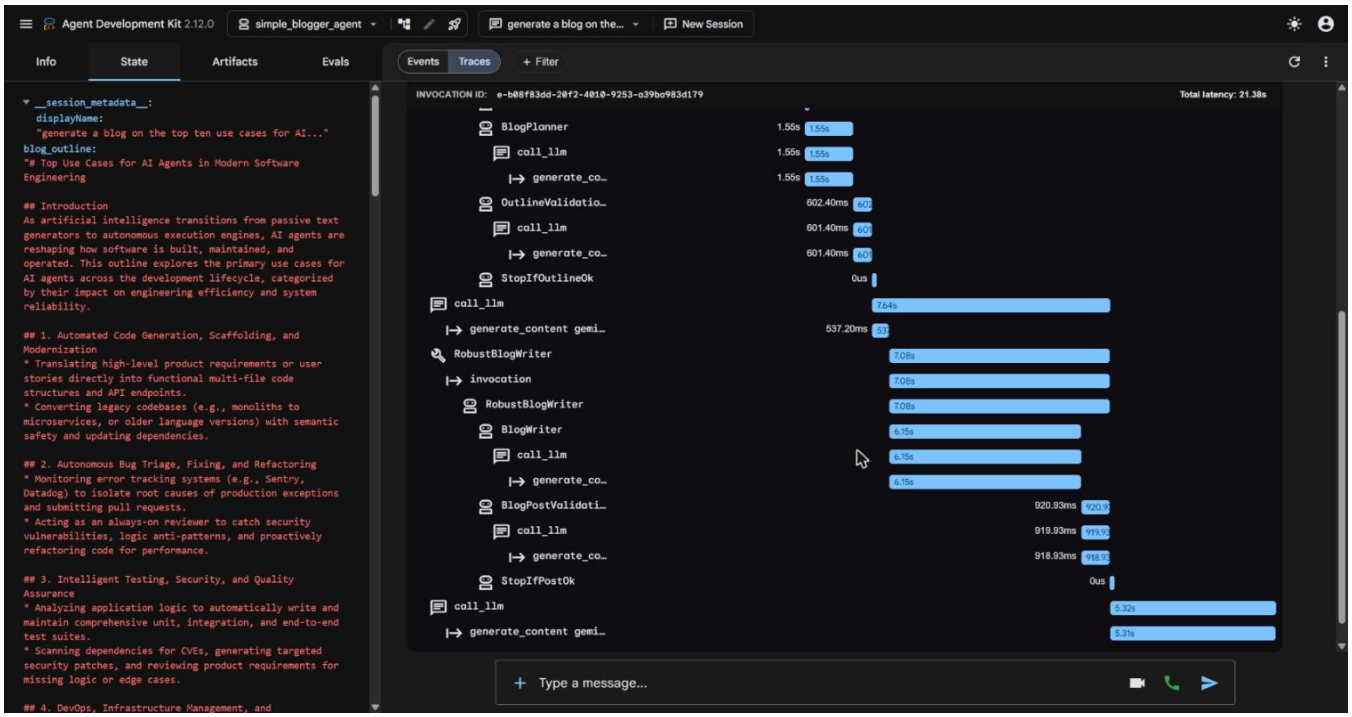
ADK Dev UI with simple_blogger_agent loaded (telemetry prompt declined).



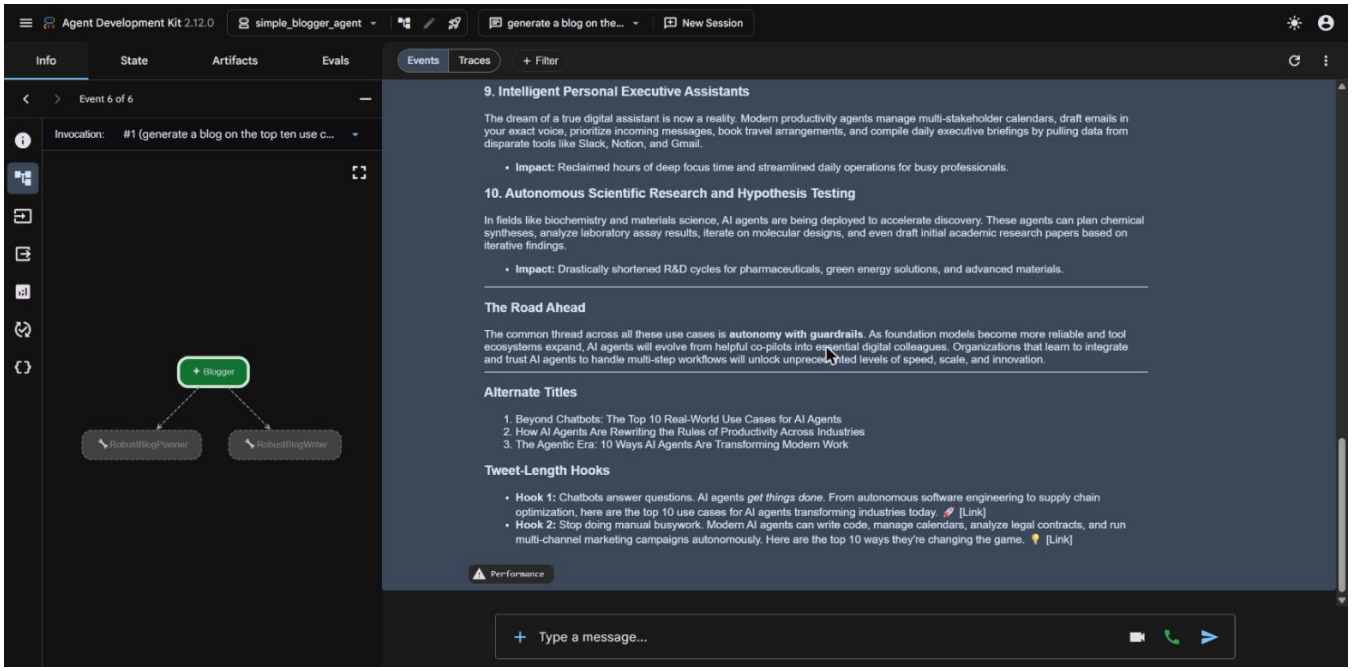
Prompt from the tutorial. Blogger calls RobustBlogPlanner, then RobustBlogWriter; each tool returns state (blog_outline, blog_post, validation_result).



Traces tab: inside RobustBlogPlanner the outline checker did not return "ok" the first time, so the loop ran BlogPlanner again. State panel on the left shows blog_outline.



Rest of the trace: the writer loop passed its check on the first try (StopIfPostOk ended the loop), then Blogger wrote the final answer. Total latency was about 21 seconds.



End of the generated post with the 3 alternate titles and 2 tweet-length hooks. The graph on the left is the same architecture as the tutorial diagram.

```
Windows PowerShell  -  loop test

PS> .venv\Scripts\python loop_test.py   # RobustBlogPlanner alone, every inner event
BlogPlanner esc= None '# Top 10 Use Cases for AI Agents: Unlocking Automation and Innovation\n\nAs artifi'
OutlineValidationChecker esc= None 'retry\n- Missing pieces: The outline must have 4-6 sections (currently it has 10 '
StopIfOutlineOk esc= False ''
BlogPlanner esc= None '# Top 5 Use Cases for AI Agents: Unlocking Enterprise Automation\n\nAs artificial '
OutlineValidationChecker esc= None 'ok'
StopIfOutlineOk esc= True ''
```

loop_test.py runs RobustBlogPlanner by itself and prints the inner events: the checker says "retry" (10 sections, rule is 4-6), the planner tries again, the checker says "ok", and StopIfOutlineOk escalates to end the loop.

```
Windows PowerShell  -  programmatic test run

PS> .venv\Scripts\python smoke_test.py   # runs root_agent with InMemoryRunner, prints each event
[  2.0s] Blogger: calls=['RobustBlogPlanner'] esc=None text=''
[  6.9s] Blogger: calls=[] esc=None text=''
[  7.5s] Blogger: calls=['RobustBlogWriter'] esc=None text=''
[ 14.9s] Blogger: calls=[] esc=None text=''
[ 18.2s] Blogger: calls=[] esc=None text='# Supercharge Your Web APIs: A Guide to Caching Strategies\n\nEvery millisec
    ond counts when building modern web applicatio'
```

A test without the browser: smoke_test.py runs the full root_agent with ADK's InMemoryRunner and prints each top-level event.

Problems I ran into

- The Gemini API key in the zip returned *402 RESOURCE_EXHAUSTED* (prepaid credits used up), so I used my own Google AI Studio key.
- *gemini-flash-latest* returned *503 UNAVAILABLE* (high demand) at the time I tested, so I set *MODEL=gemini-flash-lite-latest* in .env. The code still defaults to the tutorial's model.
- Port 8000 was already used by another local app, so I ran *adk web --port 8765*.
- One design flaw I noticed: the outline checker always wants 4-6 sections, which clashes with a "top ten" request and causes an extra retry. The root agent still delivered ten items in the final post, but a better checker would take the user's request into account.